

Choreography Saga Pattern

1. What Is the Choreography Saga Pattern?

The **Choreography Saga Pattern** is a **distributed transaction management pattern** used in **microservices architectures**.

- A **Saga** is a sequence of **local transactions**
- Each local transaction:
 - Updates its own database
 - Publishes an **event**
- Other services **listen to events** and react accordingly
- **No central coordinator** exists

In choreography:

- Services **do not call each other directly**
- Services **communicate via events**
- Each service **knows what to do next** based on the event it receives

2. Why Do We Need Choreography Saga?

In microservices:

- Each service has its **own database**
- Traditional **ACID transactions cannot span multiple services**
- Failures are inevitable

Choreography Saga provides:

- Event-driven consistency
- Loose coupling
- High scalability
- Better fault tolerance

3. Choreography vs Orchestration (Quick Context)

Aspect	Choreography	Orchestration
Coordinator	✗ None	✓ Central orchestrator
Communication	Events	Commands
Coupling	Very loose	Moderate
Complexity	Distributed	Centralized
Scalability	High	Moderate

This session focuses on **Choreography**.

4. Conceptual Example: Order Processing System

Business Scenario

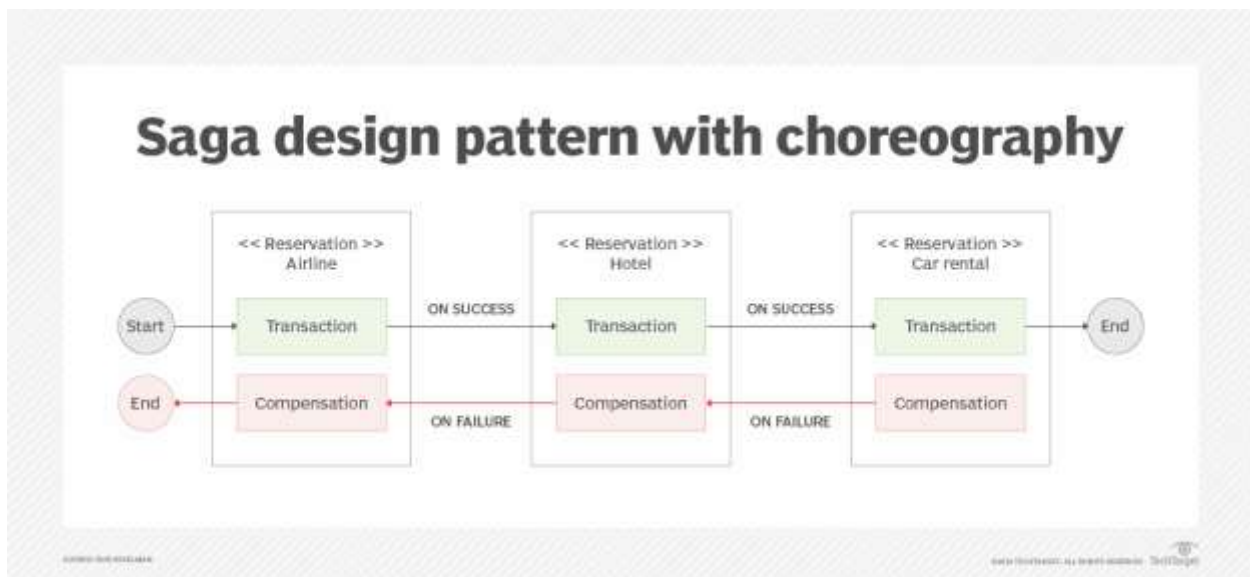
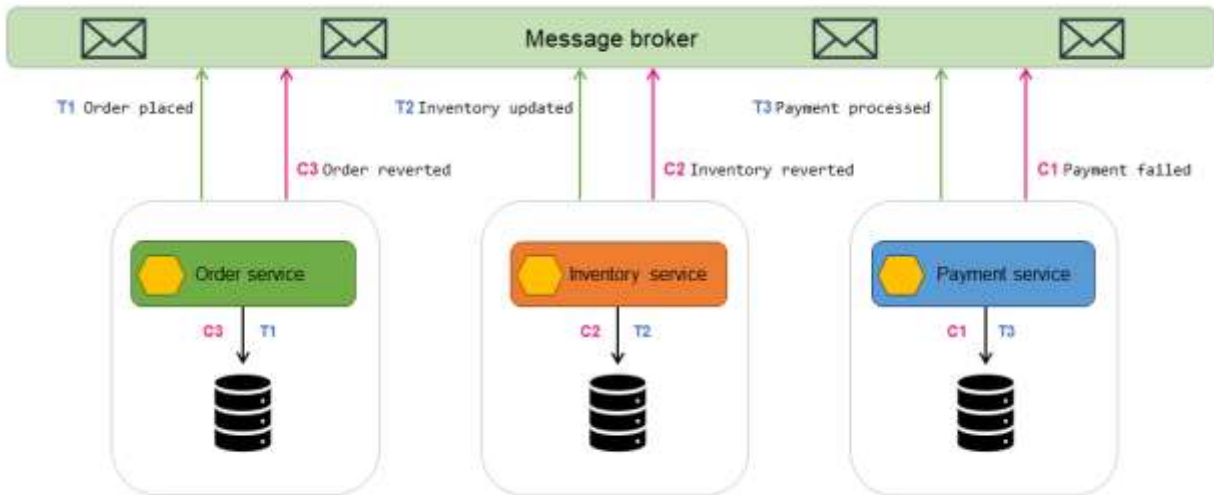
An e-commerce system with:

1. **Order Service**
2. **Payment Service**
3. **Inventory Service**

Goal:

Place an order → take payment → reserve stock

If any step fails → compensate previous steps



5. Step-by-Step Choreography Saga Flow

Step 1: Order Is Created

- Client sends **Create Order**
- **Order Service:**
 - Saves order as Pending
 - Publishes event: OrderCreated

Key point:

Order Service does **not** know who will handle the event.

Step 2: Payment Service Reacts

- **Payment Service** subscribes to OrderCreated
- It:
 - Charges the customer
 - On success → publishes PaymentCompleted
 - On failure → publishes PaymentFailed

Important:

Payment Service does not call Order Service directly.

Step 3: Inventory Service Reacts

- **Inventory Service** subscribes to PaymentCompleted
 - It:
 - Reserves stock
 - On success → publishes InventoryReserved
 - On failure → publishes InventoryFailed
-

Step 4: Order Completion

- **Order Service** subscribes to:
 - InventoryReserved
 - PaymentFailed
 - InventoryFailed

Based on event:

- InventoryReserved → Order marked as Completed
- PaymentFailed → Order marked as Cancelled

- InventoryFailed → Trigger compensation
-

6. Compensation Flow (Failure Handling)

Scenario: Inventory Reservation Fails

Steps:

1. Inventory publishes InventoryFailed
2. Payment Service listens to InventoryFailed
3. Payment Service:
 - Refunds payment
 - Publishes PaymentRefunded
4. Order Service:
 - Marks order as Cancelled

No rollback

Instead → **compensating transactions**

7. Event Flow Summary

OrderCreated



PaymentCompleted / PaymentFailed



InventoryReserved / InventoryFailed



OrderCompleted / OrderCancelled

Each service:

- Acts independently
- Reacts to events

- Emits new events
-

8. Key Characteristics

- Event-driven architecture
 - No central coordinator
 - Eventually consistent
 - Services are loosely coupled
 - Uses compensating actions instead of rollback
 - Suitable for large-scale microservices
-

9. Advantages

- High scalability
 - Loose coupling
 - Natural fit for message brokers (Kafka, Azure Service Bus, RabbitMQ)
 - No single point of failure
-

10. Challenges

- Harder to visualize flow
 - Debugging is complex
 - Requires good event design
 - Needs proper monitoring and correlation IDs
-

11. When to Use Choreography Saga

Use it when:

- System is **event-driven**
- Services must remain **independent**

- High scalability is required

Avoid it when:

- Business flow is very complex
 - Strict control and visibility are required (use orchestration instead)
-

12. One-Line Student-Friendly Definition

Choreography Saga is a pattern where microservices coordinate a distributed transaction by **publishing and reacting to events**, without any central controller.
